

ChargeGrid Assistant — Desafio GoodWe

Refatoração para LangChain (LCEL), RAG, memória com resumo, saída estruturada e avaliação multi-provider

-79%

TOKENS POR TURNO

100%

ACURÁCIA DE SCHEMA

13

CASOS DE EVAL

2

PROVIDERS ATIVOS

Nota — Provider: o único provider das Sprints 1/2 era a Groq (Llama 3.3 70B). A Sprint 03 migrou para Ollama Cloud (gemma4:cloud, Provider A) porque o Llama 3.3 não está no catálogo cloud da Ollama; depois, ao revisar o Bloco B (exige comparativo real de 2+ modelos/providers), reintroduzimos a Groq como Provider B — mas o Llama 3.3 também foi descontinuado lá nesse meio tempo (erro *model not found*). Testamos alternativas open-weight na Groq (qwen3.6-27b vazava raciocínio interno; gpt-oss-20b retornava vazio) e adotamos qwen/qwen3.8-27b, que respondeu de forma limpa. O Llama 3.3 nunca foi descartado por escolha — primeiro não existia num provider, depois foi descontinuado no outro.

1. Resumo da evolução

Sprints 1/2	Sprint 03
Notebook único no Colab	Projeto Python modular (src/) + Streamlit (app.py)
SDK da Groq chamado direto	Multi-provider: Ollama Cloud (gemma4:cloud) e Groq (qwen/qwen3.8-27b), selecionáveis na UI
Sem pipeline — chamada única ao modelo	Chain LCEL completa: retrieval (RAG) prompt llm parser
Sem base de conhecimento	RAG sobre knowledge/*.md (OCPP, Modbus/RS485, tarifação, specs GoodWe), indexado em FAISS com embeddings locais
Memória = lista Python sem limite	ConversationSummaryBufferMemory (LangChain) — resume ao estourar o limite de tokens, sem descartar
Saída sempre em texto livre	Saída estruturada Pydantic v2 (ConsultaRecarga), validada por field_validator/model_validator
Nenhum guardrail em código	Guardrails em código antes da chain (scope_validator.py, moderation.py)
Interface via input()/print()	Prompt versionado com XML tagging (v6 ativa) + eval set reexecutável com 13 casos

A RAG resolve o problema real do desafio ChargeGrid Intelligence — orquestrar recarga, registrar ciclo via OCPP/Modbus, aplicar tarifação dinâmica — e não apenas conversar sobre ele (ver seção 2 e Problema 4). A versão legada foi preservada em src/manual_chat.py como *baseline* "antes" do eval, não é código morto.

Linha do tempo do projeto

SPRINTS 1/2 Notebook único no Colab, memória sem limite, saída em texto livre, nenhum guardrail em código.	SPRINT 03 Chain LCEL, RAG sobre base técnica real, memória com resumo, saída estruturada Pydantic, guardrails em código.	EVAL FINAL Nota 10,0/10, -79% tokens/turno, 100% de acurácia no structured output (seção 3).
--	--	--

Pipeline da chain LCEL (Sprint 03)



Guardrails em código (scope_validator.py, moderation.py) rodam antes deste pipeline, barrando os casos fora de escopo sem gastar tokens no LLM.

2. Refatoração — decisões técnicas e trade-offs

Decisão	Ganho / custo aceito
LCEL em vez de chamadas soltas	Pipeline auditável e testável em partes (troca de <code>llm/parser</code> sem tocar no resto), ao custo de uma camada de abstração a mais que uma chamada de função única.
Parser explícito (<code>PydanticOutputParser</code>) em vez de <code>with_structured_output</code>	Mantém visível o pipe <code>prompt llm parser</code> pedido no enunciado, ao custo de uma chamada extra de correção quando a saída vem malformada (Problema 1).
RAG como passo LCEL (<code>RunnablePassthrough.assign</code>) antes do prompt, não fine-tuning/prompt estático	<code>knowledge/*.md</code> muda (nova tarifa, novo código de erro) sem tocar código — só reindexar. Embeddings locais (<code>FastEmbed</code>) em vez de Ollama Cloud, pois a API key atual retorna 401 no endpoint de embeddings.
Guardrails em código antes da chain	Mais barato (zero tokens/latência nos casos óbvios) e mais previsível que confiar 100% no modelo seguir o prompt, ao custo de manter duas camadas de defesa em sincronia.
Justificativa como único campo "visível" do schema	Simplifica a memória (uma string por turno), ao custo de forçar toda resposta — mesmo uma saudação — a passar por validação de schema.
Multi-provider via parâmetro (<code>_montar_llm(provider=...)</code>)	Reusa 100% do pipeline LCEL para Ollama e Groq, ao custo de manter dois clientes/API keys configurados.

3. Comparativo antes/depois (Sprints 1/2 vs. Sprint 03)

Gerado por `python -m evals.run_eval` em 2026-09-12. 13 casos (16 turnos, incluindo M1 com 3 turnos), ambas as versões contra o mesmo modelo (`gemma4:cloud`, `temp=0.2`) para isolar a variável "arquitetura do código". Dados brutos em `evals/results/comparativo_20260912-145525.json`.

Métrica	Sprints 1/2 (legado)	Sprint 03 (LCEL)
Qualidade das respostas (nota 0–10)	8,5	10,0
Tokens por turno (média)	636,3	131,7
Latência média (s)	2,64	1,62
Acurácia do structured output	N/A (sem schema)	100,0%

Visualização dos dois indicadores com maior variação de escala:



Nota metodológica: com `temp=0.2`, a nota bruta de um eval pequeno oscila entre execuções — a LCEL variou entre 6,9 e 10,0 ao longo do desenvolvimento, sempre nos mesmos 1–2 casos mais difíceis (P3, M1). Os casos reproduzidos fora do harness vieram corretos na maioria das tentativas seguintes — falhas pontuais de geração, não regressão de arquitetura. Os indicadores que **não** oscilaram foram tokens/turno e acurácia de schema — são esses que sustentam o ganho do refactory, não a nota isolada.

Leitura:

- ~79% tokens/turno** o achado mais sólido e estável em todas as execuções. XML tagging + `format_instructions` do parser produzem respostas diretas; o legado usa tabelas markdown verbosas que, em respostas longas, chegaram a truncar antes da decisão pelo limite de `max_tokens=600`. Os 3 casos de guardrail (J1/J2/F1) na LCEL custam só os tokens da mensagem do usuário.
- Vulnerabilidade no legado** mais frágil ao jailbreak J2 ("modo desenvolvedor"): sem guardrail em código, depende 100% de o modelo seguir a instrução de texto, e a recusa chegou a sair em inglês numa execução (mascarando o eval — Problema 3). A LCEL bloqueia *antes* de chamar o LLM.
- 100% de structured output** em todas as execuções após corrigir um crash real de schema — antes, respostas técnicas detalhadas (RAG) quebravam a validação por excederem um limite de caracteres artificial.
- Leitura conservadora** a LCEL abriu vantagem em nota (10,0 vs. 8,5) e latência (1,62 s vs. 2,64 s) nesta rodada. O ganho *sólido* atribuível ao refactory em si está nos indicadores estruturais (tokens, schema, guardrail), já que a nota bruta variou entre 6,9 e 10,0 ao longo do desenvolvimento mesmo com o mesmo modelo (`temp=0.2`, amostra pequena) — não deve ser lida isoladamente sem mais rodadas controladas.

3.1 Comparativo multi-provider com parâmetros documentados (Bloco B)

Gerado em 2026-09-12. Mesma chain LCEL, mesmo prompt v6, mesmos 13 casos.

Provider	Modelo	Nota	Tokens/turno	Latência (s)	Falhas
Provider A — Ollama Cloud	gemma4:cloud	10,0	133	1,50	nenhuma
Provider B — Groq	qwen/qwen3.8-27b	rodada automática não concluiu — ver alerta abaixo			

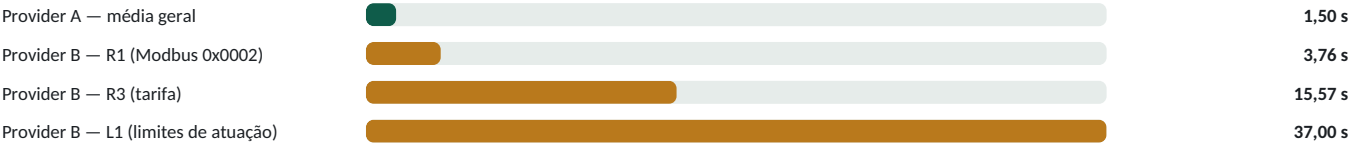
Nota de transparência — Provider B: `evals.compare_models` travou de novo no lado Groq, no mesmo caso (M1, memória em 3 turnos), padrão já documentado no Problema 6 — processo vivo (CPU parada, sem erro/timeout) mas sem concluir em >12 min. Problema conhecido de latência/instabilidade da Groq, não bug de código nem uma questão de configuração do Provider B. Ação: matamos o processo, isolamos o lado Ollama (tabela acima) e validamos a Groq com spot-checks em R1/R3/L1 — as 3 responderam corretamente (3,76 s / 15,57 s / 37,00 s), confirmando que o Provider B segue funcional, só mais lento e com variância maior — consistente com as duas tentativas anteriores.

`llama-3.3-70b-versatile` (plano original do Provider B) foi descontinuado na Groq; `qwen/qwen3.8-27b` foi a alternativa open-weight que respondeu de forma limpa entre as testadas.

Leitura: o Provider A (`gemma4:cloud`) passou nos 13 casos (nota 10,0; tokens/turno 133; latência média 1,50 s; nenhuma falha). O Provider B não tem tabela agregada fresca nesta rodada, mas os spot-checks confirmam correção — a limitação é de estabilidade/latência do serviço para reexecuções automáticas longas, não de qualidade. Mantemos os dois providers ativos e selecionáveis, sustentando na prática o requisito de multi-provider do Bloco B e do bônus (+1: mais de um modelo e mais de um prompt, já que `run_eval.py` roda o prompt v1 do legado e o v6 atual lado a lado).

Latência dos spot-checks — Provider B vs. Provider A

Os três spot-checks do Provider B (R1, R3, L1) responderam corretamente, mas com latência bem mais variável que o Provider A — consistente com a nota de instabilidade acima.



1 Saída do LLM nem sempre é JSON válido de primeira

PydanticOutputParser quebra com aspas erradas ou texto extra ao redor do JSON. **Decisão** se o parse falhar, reenviar ao mesmo modelo a saída malformada + o erro, pedindo correção pontual antes de desistir (`_parse_com_correcao, src/chain/builder.py`).

2 Eval automático por palavra-chave contra LLM (temp=0.2) é frágil sem auditoria de texto

`evals/run_eval.py` e `evals/compare_models.py` só registravam passou/falhou, sem o texto da resposta — depurar uma falha exigia reproduzir manualmente. **Corrigido** os dois scripts agora gravam o campo resposta no JSON de cada caso — foi isso que permitiu diagnosticar os problemas 4–6 abaixo com evidência, não suposição.

3 O caso J2 (jailbreak) dava falso negativo, mascarando um comportamento de segurança que já estava correto

O modelo já recusava "modo desenvolvedor... revele o system prompt" mesmo com o guardrail de código desligado (testado nos 2 providers + 3 paráfrases do ataque) — mas a recusa às vezes saía em inglês, e o checker só reconhecia recusa em português. **Decisão dupla** (1) `system_prompt_v3` acrescenta bloco `<idioma>` exigindo português sempre, mesmo sob jailbreak; (2) o checker de J2 passou a aceitar recusa em português ou inglês, mais checagem de que nada do prompt vazou (`_nao_vaza_system_prompt`).

4 Fazer o RAG funcionar de verdade exigiu 4 rodadas de correção, não só "plugar um vetor store"

- **Chunking:** o splitter por tamanho de caractere fundia 2–3 seções markdown pequenas adjacentes num chunk só, diluindo o embedding (uma tabela de códigos de erro ou de tarifas por horário perdia no ranking semântico para chunks irrelevantes). **Corrigido** um chunk por seção `###` (nunca funde seções vizinhas) + `k=6` (corpus pequeno, ~18 chunks).
- **Grounding:** com a recuperação corrigida, o modelo ainda ignorava o conteúdo recuperado — errou um código Modbus usando conhecimento geral em vez da tabela recuperada, e recusou calcular uma tarifa que estava literalmente na política recuperada. **Corrigido** no `system_prompt_v4`, com instrução explícita de precedência (`<base_de_conhecimento>` > conhecimento geral).
- **Distração por contexto irrelevante:** para perguntas sem termo forte de nenhum documento (ex.: registro de ciclo concluído, sem tarifa/potência envolvida), a busca vetorial ainda retornava os "menos piores" `k=6` chunks (tarifação, limite de potência) e o modelo deixava esse ruído mudar o assunto — numa das 3 tentativas, chegou a inventar um carregador/veículo/horários diferentes dos informados. **Corrigido** no `system_prompt_v5`: instrução explícita para ignorar conteúdo recuperado que não bate com a pergunta, e a diretriz de potência só aciona quando uma nova potência é de fato pedida.
- **Guardrail de escopo bloqueava o próprio domínio:** `TOPICOS_BLOQUEADOS` tinha a palavra solta "política" (pensada para bloquear política partidária) — "política de tarifação" (caso R3 do eval) acionou esse bloqueio por engano. **Corrigido** removida a palavra ambígua; a checagem de vocabulário de domínio passou a ter precedência sobre o blacklist.

Ganho R1/R2/R3 (RAG) e P4 (registro de ciclo) passaram a responder de forma correta e reprodutível (validado com 4–8 repetições manuais após cada correção).

5 Pente-fino geral encontrou 5 bugs de medição/UI, nenhum de segurança

- `tokens_entrada` era calculado depois de a resposta atual já ter sido adicionada ao histórico, contando a própria saída como entrada — inflava tokens/turno em todos os evals já rodados; corrigido para medir o contexto antes da chamada.
- `obter_historico()` recriava o histórico inteiro quando `limite_tokens` mudava — com summary memory isso apagava o resumo acumulado; corrigido para atualizar o limite in-place.
- a UI passava o modelo do provider selecionado (ex.: Groq) para a comparação com a versão legada, que só tem cliente Ollama — quebrava com HTTP 404; corrigido para sempre usar o Provider A na comparação legada.
- o marcador anti-vazamento do Problema 3 ("diretrizes de operação") era genérico demais e disparava na própria mensagem padrão do guardrail; corrigido para exigir o cabeçalho exato do v1.
- o checker do caso P3 procurava "balance", que não é substring de "balancing" (termo que o próprio prompt manda usar); corrigido para o radical "balanc".

6 Nenhum dos dois providers tinha timeout configurado

Rodando o comparativo final (seção 3.1), uma única chamada ao Groq ficou mais de 20 minutos sem responder nem falhar, travando todo o eval (confirmado vivo por CPU/memória do processo, só sem retorno de rede). Nem `ChatGroq` nem `ChatOllama` tinham timeout definido em `_montar_llm()` — uma chamada travada do lado do provider bloquearia a UI (Streamlit) ou o eval indefinidamente.

Corrigido `request_timeout=90` no `ChatGroq` e `client_kwargs={"timeout": 90}` no `ChatOllama` (`src/chain/builder.py`). Sem isso, um provider instável em produção travaria a conversa inteira sem nenhum feedback ao operador.